
DineFlow

How it is built, how it is kept safe, how it is run —
and what is not finished yet.

Who this is for

Whoever runs, deploys or changes DineFlow

Not for clients

It states the weaknesses plainly, on purpose

Measured

22 September 2026 · against the live database

1 · The system at a glance

66 tables · 55 per-property, 11 shared	199 database functions, 179 security definer	71 routes in the web app	69 migrations, applied in order
--	--	------------------------------------	---

PIECE	WHAT IT IS
apps/web	Next.js 15, App Router. Server components, server actions. The whole product
apps/desktop	Electron shell pointing at the hosted app. Optional
apps/mobile	Expo. Scanning and a few on-the-floor screens. Optional
packages/shared	Constants, zod schemas, billing and tax maths. Used by both server and browser
supabase/migrations	Every schema change, numbered, applied once, recorded
Supabase	PostgreSQL 17.6 · Auth · Realtime · Storage · 45 MB, AWS ap-south-1 (Mumbai)

Business logic lives in the database, not the app. Placing an order, generating a bill, checking in, the night audit — each is a Postgres function called over RPC. The rules hold whichever client calls them, and a bug in one screen cannot produce an invalid bill.

2 · How one property is kept away from another

This is the most important property of the system. It is enforced by Postgres, not by application code that could be forgotten.

- Every per-property table carries `restaurant_id`.
- Row-level security is **on for all 55** of them, with 56 policies between them.
- Every policy scopes rows to `auth_restaurant_id()` — the property on the caller's own token — or, where the row belongs to a person rather than a property, to `auth.uid()`.
- Write policies carry `WITH CHECK`, so a forged `restaurant_id` is refused rather than stored.
- A trigger stamps the property on insert, so a client never has to supply it correctly.

Verified, not assumed

Signed in as a real owner of one property, reaching for another, on the live database:

ATTEMPT	RESULT
Read another property's rows, across all 55 tables	0 rows visible
Update another property's menu	0 rows changed
Insert a row into another property	Refused by the database
Resolve the session	Own property only

Because the fence is in the database, this holds for the web app, the phone app, a direct API call, or anything written later. There is no path around it short of the service key.

3 · Performance, measured

Where the time goes

Round trip to the database from the office: **34 ms**. Nearly every screen's whole query set completes within that — the queries themselves cost almost nothing, and what you wait for is distance.

MEASUREMENT	RESULT
A page's full query set	32–43 ms (the round trip is 34 ms of it)
Index plans on hot queries	< 0.1 ms, no sequential scans
Tables lacking a <code>restaurant_id</code> index	0
14× the data (205 properties, 68,669 order lines)	worst change +1 ms

The load test matters more than the timings. A real property was cloned 200 times and the screens re-timed, as a real owner, through row-level security. Nothing measurably slowed. The system is not going to degrade as properties are added.

What the browser downloads

SCREEN	FIRST LOAD JS
Shared by every page	103 kB
Kitchen	174 kB
Front desk	158 kB
Housekeeping	155 kB

Two things keep these down: the shared package declares itself side-effect free, so zod does not reach the browser; and Supabase's realtime client is fetched after paint, so screens that never open a socket never download it.

The setting that decides everything

`vercel.json` pins the server to **bom1 (Mumbai)**, beside the database. Without it Vercel runs in Washington DC and every dynamic page makes its round trips across the world — roughly 200 ms each, twice per page. Most pages are dynamic, so this is not a detail.

4 · Security posture

AREA	POSITION
Passwords	Held by Supabase Auth, hashed. No password column on any per-property table
Tenant isolation	Row-level security on all 55 per-property tables
Definer functions	179 of 199; 0 without a pinned <code>search_path</code>

AREA	POSITION
One-time codes	<code>password_otp</code> s , <code>contact_otp</code> s — RLS on, no policy, reachable only through definer functions
Payment secrets	<code>payment_gateways</code> — owner-scoped, never part of the session row sent to screens
Stale sessions	A refused refresh token is cleared rather than retried forever

Known and accepted: temporary passwords are stored in clear text.

`temp_passwords.password` holds a readable password so Master control can give it to an owner. Mitigations are real — the table has **no RLS policy at all**, so no signed-in user can reach it; every path goes through a function gated on `is_platform_admin()` ; and the value is blanked on expiry, on use, or when the owner sets their own.

The residual risk is that these sit in plain text **in database backups**, and a leaked service key yields working logins. If that is not acceptable, store only the fact that a password was issued, or encrypt it. It is a deliberate trade, not an oversight.

5 · Changing the database

```
node scripts/migrate.mjs
```

Applies anything in `supabase/migrations` not already recorded in `_dineflow_migrations`, in filename order, once each.

The rules that keep this safe

1. **Always dry-run first** — run the file inside a transaction that is rolled back, against the live database, and assert the outcome. Every migration in this project was.
2. **Additive, not destructive.** Add columns with defaults; do not repurpose an existing one.
3. **Never add a defaulted argument to an existing function** without dropping the old signature first — two overloads make every PostgREST call ambiguous, and the app breaks at once.
4. **Pin `search_path`** on every `security definer` function.
5. **Index `restaurant_id`** on any new per-property table. It is the most-run predicate in the system.

There is no undo. A migration against production is permanent, and the free Supabase tier keeps no backups.

6 · Testing and CI

Vitest, with `fake-indexeddb`, driving the real offline store; only the network call is replaced.

```
pnpm typecheck && pnpm lint && pnpm test && pnpm build
```

CI runs the same four on every push. The 8 tests cover the outbox: backoff, the hold once retries run out, an immediate hold on bad data, held jobs not being retried, the duplicate case, send again, discard, and the ordinary path.

The tests were checked against the old code and four of them failed on it. A test that passes either way is not worth keeping. Apply the same standard to anything added.

Coverage beyond the outbox is thin — see the gap list.

7 · Distribution

The browser install is the primary path on every platform: one click on Windows, Mac and Android; three taps on iOS, which is Apple's minimum. It downloads nothing, raises no warning, and follows the site on every deploy. `/get` detects the device and the window it is in, including in-app browsers that cannot install at all.

PATH	COST OF THE ALTERNATIVE
Windows <code>.exe</code>	Unsigned → SmartScreen warning. A certificate is ~\$200–400/year
Android <code>.apk</code>	Requires “install from unknown sources” — more friction, not less
iOS App Store	\$99/year, review on every release

```
node scripts/set-app-url.mjs https://app.example.in
```

Points the desktop and phone builds at one address. The website derives its own from each request.

8 · Runbooks

Deploy

1. Gates green locally.
2. Push; CI repeats them.
3. Deploy. Migrations do **not** run automatically — apply them deliberately.

A till is stuck “working offline”

1. Check the property's Wi-Fi, then `/api/health?deep=1`.
2. Anything unsendable is held and listed on the device, not lost.
3. Once the line is back it flushes on its own; **Send again** forces it.

Everyone is being signed out

Expected after an auth change. The middleware clears a refused token rather than retrying it. If it persists, check the Supabase project is not paused.

A screen is slow

1. Confirm the deployment region is still `bom1`.
2. `explain (analyze)` the query; look for a sequential scan.
3. Check `restaurant_id` is indexed on any new table.

9 · What is not finished

Stated plainly so it can be planned rather than discovered.

GAP	CONSEQUENCE
No error monitoring	A failure in someone else's restaurant reaches you by telephone. Sentry's free tier would close this
Thin test coverage	Only the outbox is covered. Billing, GST and the folio — the parts that handle money — are verified by hand
AI features never executed	<code>ANTHROPIC_API_KEY</code> has never been set; 13 files depend on it. Gated off, so they should stay hidden rather than fail
Signed-in screens never driven end to end	Verified by types, queries, build and rendering — not by use
Free Supabase tier	No daily backups, no point-in-time recovery, pauses when idle. Not acceptable for live properties
Deployment not public	Protection on, preview URL. Nothing can be installed until this changes
Never load-tested concurrently	Data volume is proven; many simultaneous devices are not
Unsigned installers	Windows warns on every install

Order of work, if it were mine: backups first — they protect what already exists. Then error monitoring, then tests around billing and tax, then the public deployment.

10 · Where things are

PATH	WHAT IS IN IT
apps/web/app/(app)/	Every signed-in screen, one folder each
apps/web/lib/offline/	The outbox, the store, and its tests
apps/web/lib/auth.ts	Session resolution; every page begins here
apps/web/middleware.ts	What is public, and dead-session handling
packages/shared/src/	Billing, tax, schemas, constants
supabase/migrations/	The schema's whole history
scripts/migrate.mjs	Applies migrations
scripts/set-app-url.mjs	Points the built apps at one address
GO-LIVE.md	Test deployment to production, step by step